

# Reproducibility Sheet — CS7641 OL Report

## Optimization & Uncertainty in Learning, Summer 2026

Timothy Leung  
tleung37@gatech.edu gtID 904150400  
Georgia Institute of Technology, OMSCS

### 1. Overleaf project (read-only)

**Overleaf project:** [overleaf.com/project/6a11e4cebb9a5dd9d3751c28](https://overleaf.com/project/6a11e4cebb9a5dd9d3751c28)

The project is hosted on Georgia Tech Enterprise Overleaf ([overleaf.gatech.edu](https://overleaf.gatech.edu)); the read-only link above grants the grader full revision-history access without write permission. Revision timeline is intact from first commit through final submission build.

### 2. GitHub commit hash (single SHA)

**Final commit SHA:** bcadfceebf5342f7775caf9aba20f8e86cec3c3e

**Repository:** [github.gatech.edu/tleung37/CS7641\\_ml\\_report2](https://github.gatech.edu/tleung37/CS7641_ml_report2) (GT Enterprise, private)

**Branch:** main

**Tag:** v1.0-submission

The repository contains every script that produces every figure, every table, and every number in the report. Branch and commit history are intact and available for review.

### 3. Exact run instructions (standard Linux machine)

#### 3.1 Environment setup

Tested on Ubuntu 22.04 LTS, Python 3.11, R 4.3. Approximate end-to-end runtime: 35–45 minutes on a 4-core CPU laptop (CUDA GPU reduces to ~8 min).

```
# Clone the repository
git clone https://github.gatech.edu/tleung37/CS7641_ml_report2.git
cd CS7641_ml_report2
git checkout bcadfceebf5342f7775caf9aba20f8e86cec3c3e

# Python environment
python3.11 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt

# R environment (for rsanity/ audit; not required for grading main report)
Rscript -e 'install.packages(c("bookdown","yaml","jsonlite","dplyr","ggplot2",
                             "knitr","kableExtra","nnet","caret"),
          repos="https://cloud.r-project.org")'
```

## 3.2 Commands to reproduce every figure and table

```
# (a) Run the full pipeline end-to-end.
#     Reads configs/config.yaml, writes results/{logs,tables,figures}/
python run_pipeline.py

# (b) Compile the OL report PDF (twice for cross-references).
cd report
pdflatex -interaction=nonstopmode OL_Report.tex
pdflatex -interaction=nonstopmode OL_Report.tex
cd ..

# (c) Rerun the R sanity check (independent audit of Python results).
Rscript -e 'rmarkdown::render("rsanity/sanity_check_OL.Rmd", output_dir="rsanity")'

# (d) Compile this reproducibility sheet.
cd repro
pdflatex -interaction=nonstopmode REPRO_OL.tex
```

## 3.3 Data paths

`sklearn.datasets.fetch_covtype()` is called by `src/data_loader.py` on first run; the dataset (~75 MB) caches to `.cache/covtype/`. No manual download is required; the script will fetch from UCI if no cache is present.

## 3.4 Random seeds

Primary seed: `seed = 7641` (course number; same as SL Report), set in `configs/config.yaml` and propagated to `numpy.random.seed`, `random.seed`, `torch.manual_seed`, and CuDNN determinism flags. Multi-seed stability runs (Parts 2 and 3) additionally use seeds `{42, 123, 7}`. Re-running on a clean checkout reproduces every reported number to within floating-point precision on the same Python and `scikit-learn`/`torch` versions.

## 3.5 Output landing locations

- `results/logs/sl_sgd_baseline.json` — SL PyTorch SGD baseline metrics (the reference point for all OL comparisons).
- `results/logs/{rhc,sa,ga}.json` — Part 1 RO progress and final test metrics per algorithm.
- `results/logs/part2-{optimizer}_seed{s}.json` — Part 2 ablations; one file per optimizer  $\times$  seed.
- `results/logs/part3-{regularizer}_seed{s}.json` — Part 3 regularization study; one file per condition  $\times$  seed.
- `results/logs/heatmap_alpha_beta1.json` — Adam  $(\alpha, \beta_1)$  sensitivity grid.
- `results/tables/final_metric_table.csv` — the headline leaderboard across all methods.
- `results/figures/*.pdf` — all 10 publication figures.
- `report/OL_Report.pdf` — the compiled main report.
- `rsanity/sanity_check_OL.pdf` — the R audit output.

## 4. SL Continuity Contract

The OL Report reuses the Covertypes sample, preprocessing pipeline, split, and PyTorch backbone from the SL Report without modification. The table below confirms each component is unchanged.

Table 1: SL  $\rightarrow$  OL continuity verification.

| Component     | SL value                                          | OL value       | Changed? |
|---------------|---------------------------------------------------|----------------|----------|
| Dataset       | Covertypes $n=20k$                                | Same           | No       |
| Seed          | 7641                                              | 7641           | No       |
| Split         | 60/20/20 stratified                               | Same           | No       |
| Preprocessing | StandardScaler (train-only)                       | Same           | No       |
| Architecture  | FC(256) $\rightarrow$ FC(128) $\rightarrow$ FC(7) | Same           | No       |
| Test set      | Stratified 4k rows                                | Same partition | No       |

## 5. Full-data and sampled-data EDA (carried over from SL)

The Covertypes dataset contains 581,012 rows, 54 features (10 continuous + 4 wilderness-area + 40 soil-type binary indicators), and seven cover-type classes.

Table 2: Full Covertypes class distribution. Class-frequency ratio  $\approx 103:1$ .

| Class               | Count   | Share (%) |
|---------------------|---------|-----------|
| 1 Spruce/Fir        | 211,840 | 36.46     |
| 2 Lodgepole Pine    | 283,301 | 48.76     |
| 3 Ponderosa Pine    | 35,754  | 6.15      |
| 4 Cottonwood/Willow | 2,747   | 0.47      |
| 5 Aspen             | 9,493   | 1.63      |
| 6 Douglas-fir       | 17,367  | 2.99      |
| 7 Krummholz         | 20,510  | 3.53      |
| Total               | 581,012 | 100.00    |

Table 3: Sampled-data ( $n=20,000$ ) class distribution — same proportions as SL.

| Class               | Count  | Share (%) |
|---------------------|--------|-----------|
| 1 Spruce/Fir        | 7,292  | 36.46     |
| 2 Lodgepole Pine    | 9,751  | 48.76     |
| 3 Ponderosa Pine    | 1,231  | 6.16      |
| 4 Cottonwood/Willow | 95     | 0.47      |
| 5 Aspen             | 327    | 1.64      |
| 6 Douglas-fir       | 598    | 2.99      |
| 7 Krummholz         | 706    | 3.53      |
| Total               | 20,000 | 100.00    |

A subsequent stratified 60/20/20 split yields a 12,000-row training partition, a 4,000-row validation partition, and a 4,000-row test partition; the test set is the same sealed partition as the SL Report.

## 6. Exact sampling procedure

Identical to SL Report §6: `StratifiedShuffleSplit(n_splits=1, train_size=20000, random_state=7641)` on the full 581k rows, followed by a second `StratifiedShuffleSplit(test_size=0.20, random_state=7641)` on the 20k sample, then a third `StratifiedShuffleSplit(test_size=0.25, random_state=7641)` on the 16k remainder to produce the internal 12k train / 4k val split.

**Standardisation.** `StandardScaler` fit on the 12,000-row training partition only; applied to validation and test. The 44 binary indicators pass through unchanged. No test row is seen by any preprocessing step until final evaluation. Implemented in `src/data_loader.py`.

## 7. Compute accounting

### 7.1 Gradient-based methods (Parts 2 & 3)

One minibatch backward pass = 1 gradient evaluation (GE) = 1 optimizer update. Batch size 256,  $n_{\text{train}}=12,000$ :  $\lceil 12000/256 \rceil = 47$  GE/epoch. 80 epochs = 3,760 GEs per run (before early stopping). All GE counts logged under `grad_evals` in each JSON log.

### 7.2 Randomized optimization (Part 1)

One full validation-set evaluation = 1 function evaluation (FE).

| Method | Budget                     | Count rule              |
|--------|----------------------------|-------------------------|
| RHC    | 3 restarts $\times$ 400 FE | 1 val eval = 1 FE       |
| SA     | 2,000 FE                   | 1 val eval = 1 FE       |
| GA     | 50 gen $\times$ pop 20     | 1 candidate eval = 1 FE |

## 8. RO hyperparameter disclosures

RO trainable layers: final 2 (FC(128) $\rightarrow$ BN $\rightarrow$ ReLU $\rightarrow$ FC(7) + biases). RO trainable parameter count: 1,159 (verified at runtime;  $\ll$  50k cap).

- **RHC**: 3 restarts; perturbation  $\mathcal{N}(0, 0.01^2 \mathbf{I})$ ; restart scale  $5\sigma$ ; 400 FE/restart.
- **SA**:  $T_0=1.0$ , geometric cooling  $T_t=T_0 \cdot 0.995^t$ ; Metropolis acceptance; 2,000 FE.
- **GA**: pop 20, tournament ( $k=3$ ), uniform crossover ( $p=0.7$ ), mutation  $\mathcal{N}(0, 0.01^2 \mathbf{I})$ , top-2 elitism; 50 generations. Progress reported by FE, not generation.

## 9. Adam-family optimizer disclosures (Part 2)

| Optimizer            | lr        | Other HPs                               |
|----------------------|-----------|-----------------------------------------|
| SGD (no mom.)        | 0.05      | momentum=0                              |
| SGD+momentum         | 0.05      | $\mu=0.9$ , nesterov=False              |
| Nesterov             | 0.05      | $\mu=0.9$ , nesterov=True               |
| Adam                 | $10^{-3}$ | $\beta_1=0.9$ , $\beta_2=0.999$         |
| Adam (no bias-corr.) | $10^{-3}$ | custom; omits $\hat{m}_t$ , $\hat{v}_t$ |
| Adam ( $\beta_1=0$ ) | $10^{-3}$ | $\beta_1=0$ , $\beta_2=0.999$           |
| AdamW                | $10^{-3}$ | $\lambda=10^{-4}$ , decoupled WD        |

Threshold  $\ell=0.539$  ( $= 0.90 \times$  initial val loss, SGD-no-momentum, seed=42; set from training evidence only, never from test).

Adam without bias correction update rule:

$$\begin{aligned}
 m_t &= \beta_1 m_{t-1} + (1 - \beta_1) g_t, \\
 v_t &= \beta_2 v_{t-1} + (1 - \beta_2) (g_t)^2, \\
 \theta_t &= \theta_{t-1} - \alpha m_t / (\sqrt{v_t} + \varepsilon).
 \end{aligned}$$

Custom implementation in `src/optimizer.py::AdamNoBiasCorrection`.

## 10. Regularization disclosures (Part 3)

Standard Adam fixed:  $\text{lr}=10^{-3}$ ,  $\beta_1=0.9$ ,  $\beta_2=0.999$ ,  $\varepsilon=10^{-8}$  (no weight decay unless listed).

| Condition        | Configuration                                                     |
|------------------|-------------------------------------------------------------------|
| Adam baseline    | No regularization; Adam defaults                                  |
| L2 $10^{-4}$     | <code>weight_decay=10<sup>-4</sup></code> (coupled L2, not AdamW) |
| L2 $10^{-3}$     | <code>weight_decay=10<sup>-3</sup></code>                         |
| Early stopping   | patience=10 epochs; monitored: val loss; best weights restored    |
| Dropout 0.2      | <code>nn.Dropout(0.2)</code> after each hidden layer              |
| Dropout 0.3      | <code>nn.Dropout(0.3)</code> after each hidden layer              |
| Label smooth 0.1 | <code>CrossEntropyLoss(label_smoothing=0.1)</code>                |
| Input noise 0.05 | $x \leftarrow x + \mathcal{N}(0, 0.05^2)$ , training only         |
| Best combo       | L2( $10^{-4}$ ) + early stop(p=10) + label smooth(0.1)            |

## 11. R sanity check — method and interpretation

The `rsanity/sanity_check_OL.Rmd` performs two audits:

1. **Baseline agreement.** Trains `nnet` (BFGS, size=64, decay= $10^{-3}$ , 200 maxit, 8k subsample) on the same seed-7641 stratified split and compares its test macro-F1 to the Python SL SGD baseline loaded from `results/logs/sl_sgd_baseline.json`.  $|\Delta_{\text{macro-F1}}| < 0.03$  = confirmed; 0.03–0.10 = expected BFGS-vs-SGD library difference;  $> 0.10$  = investigate. The observed gap is **−0.0371** (R `nnet` = 0.5533 vs. Python SGD = 0.5904), in the expected direction and magnitude, confirming the Python split and preprocessing are faithful. The one-sided negative bias is expected: R uses BFGS on an 8k subsample; Python uses pure SGD on 16k.

2. **Log audit.** Loads every `results/logs/part2_*.json` (21 runs) and `part3_*.json` (27 runs) — 48 files total — and checks:  $\text{macro-F1} \in [0, 1]$ , GE counts  $\geq 0$ , no corrupted JSON. All 48 runs pass. Largest seed IQR is 0.035 (stable across seeds). Delta-F1 vs. SL SGD baseline is plotted per regularization condition.

## 12. Hardware and software versions

- OS: Ubuntu 22.04 LTS (kernel 6.5)
- CPU: 4-core x86\_64; no GPU used for submission numbers (CUDA path available but not required)
- Python: 3.11.6
- Key packages: `torch 2.3.1`, `scikit-learn 1.5.0`, `numpy 1.26.4`, `pandas 2.2.2`, `matplotlib 3.9.0`, `seaborn 0.13.2`
- R: 4.3.x; `nnet 7.3`, `caret 6.0`, `bookdown 0.40`, `jsonlite 1.8`, `ggplot2 3.5`
- LaTeX: TeX Live 2025 / TinyTeX with IEEEtran conference class (`IEEEtran.cls v1.8b`)